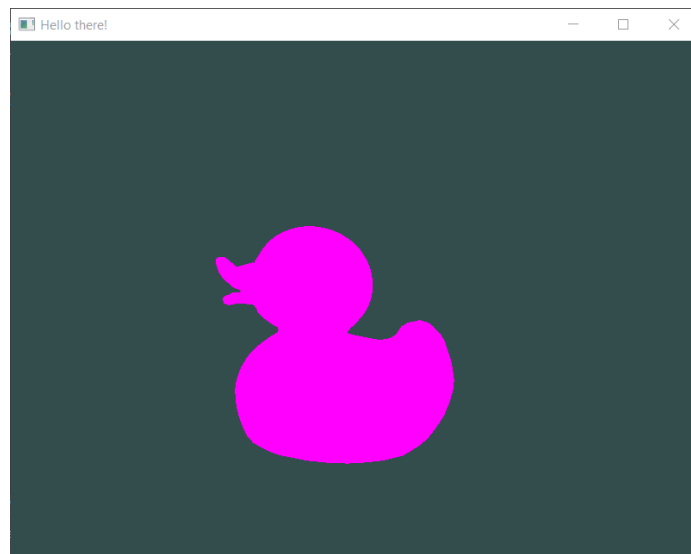


Exercise: Shading and Texturing

1 The Current Project

Make your self familiar with the current version of the project (available on moodle). Either copy the files into your project or check that your version has the same capability:

- SceneGraph (with transform logic)
- Mesh Class
- Shader Class
- Camera Class
- Gltf Model loading

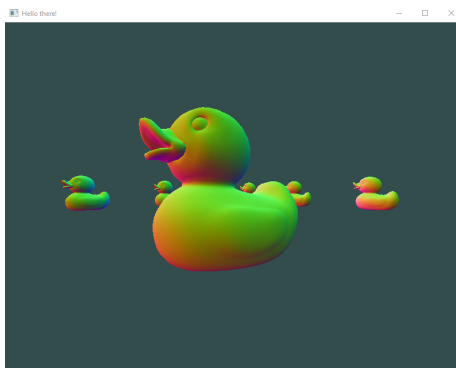


Controls:

- **W,A,S,D**: move in the xz plane
- **SHIFT,SPACE**: move on y axis
- **MOUSE_WHEEL**: change Field of View

2 The Scene

Create a small scene. Use the models provided under *asset* in the moodle course or some you find on the internet. The scene should use hierarchical *rotation*, *translation* and *scaling*.



Example scene with the duck model (texture coordinates visualized (see exercise 3))

You can also update the transforms in the render loop with sin or cos to create simple animations.

3 Visualizing the Data!

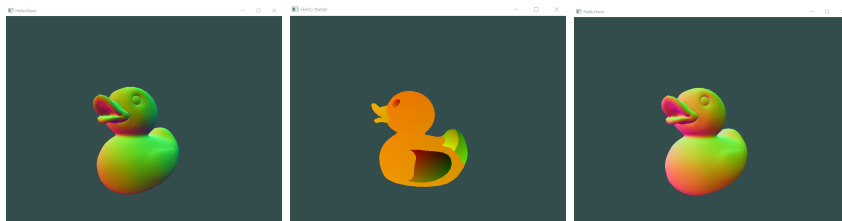
Change the *default.frag* and *default.vert* to visualize the following attributes as color:

- Texture coordinates
- Normals (in model space)
- Normals (in world space)

Classes, functions and keywords you may need in your shader code:

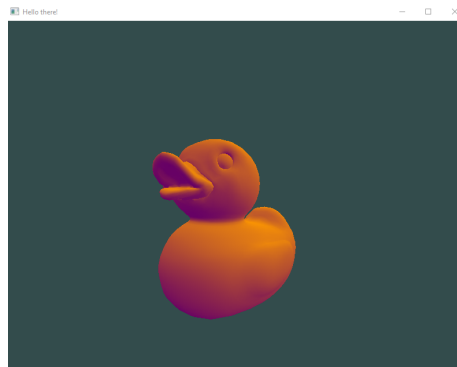
`normalize` , `uniform` , `in` , `out` , `vec3` , `vec4` , `mat4x4` .

Which attributes is shown in each of these pictures?



4 Gooch Shading!

Change the *default.frag* and *default.vert* to a simple Gooch shading using two colors of your choosing:



Gooch shading with a light orange and a dark purple

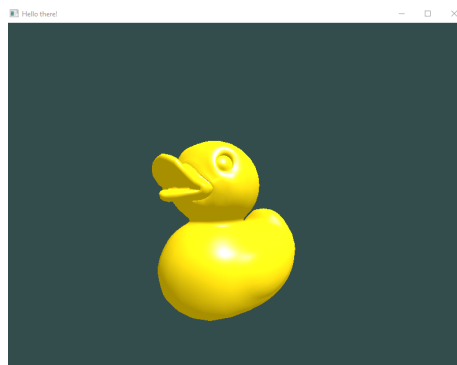
You may hard code the light direction and light and dark color. However it is more flexible if they are implemented as uniform variables.

Classes, functions and keywords you may need in your shader code:

`dot` , `mix` , `uniform` , `in` , `out` , `vec3` , `vec4` .

5 Phong Shading!

Copy the *default.frag* and *default.vert* files and change them to *phong.frag* and *phong.vert*. Then implement the phong shading model in the new shader:



You may need to add the files to your CMake configuration for them to show up in VisualStudio.

Classes, functions and keywords you may need in your shader code:

`normalize` `reflect` `max` `dot` `pow` `max` `uniform` `in` `out`
`vec3` `vec4`

6 Texture Class

Take a look at the *Texture Class* which came with the updated files (and is available in the snippets).

Please use the learn OpenGL Texture Tutorial as reference for this exercise.

a) Follow the instructions in the comments to implement the Functions!

Functions and macros you may need in your code:

```
glGenTextures    glBindTexture    GL_TEXTURE_2D    glTexParameteri
GL_TEXTURE_WRAP_S    GL_TEXTURE_WRAP_T    GL_REPEAT
GL_TEXTURE_MIN_FILTER    GL_TEXTURE_MAG_FILTER    GL_NEAREST
GL_LINEAR    GL_NEAREST_MIPMAP_NEAREST    GL_LINEAR_MIPMAP_NEAREST
GL_NEAREST_MIPMAP_LINEAR    GL_LINEAR_MIPMAP_LINEAR    GL_RGBA
GL_RGB    GL_RG    glTexImage2D    GL_UNSIGNED_BYTE    glGenerateMipmap
glDeleteTextures    glActiveTexture    GL_TEXTURE0    glEnable    glBindTexture
stbi_load    stbi_image_free    resolvePath
```

b) Change the *phong.frag* and *phong.vert* shader to read the base color from a texture

Classes, functions and keywords you may need in your shader code:

```
texture    max    uniform    sampler2D    in    out    vec2    vec3
vec4
```

c) Load a texture and bind it to the shader in the *main.cpp*

Classes, functions and keywords you may need in your code:

```
Texture    Texture :: Bind    Shader :: setVec3    Shader :: setInt
```

RESULT: now you should have an image like this:

